

ID	Nombre	Apellidos	Email	Teléfono	Fecha Nacimiento	Confirmado	Contraseña	Rol
1	María	García López	maria.garcia@example.com	600123456	1990-05-15	1	m123	miembro
2	Juan	Pérez Martínez	juan.perez@example.com	600654321	1985-03-22	1	j123	administrador
3	Ana	López Sánchez	ana.lopez@example.com	600987654	1993-07-10	0	a123	miembro
4	Pedro	Gómez Ruiz	pedro.gomez@example.com	600111222	1992-02-28	0	p123	miembro
5	Lucía	Martín Gómez	lucia.martin@example.com	600333444	2000-11-05	0	l123	miembro
6	Carlos	Díaz Fernández	carlos.diaz@example.com	600555666	1995-08-18	0	c123	miembro

ID	Nombre	Apellidos	Token	Fecha Expiración	imagen	nif_nie
1	María	García López	NULL	NULL	maria.png	12345678Z
2	Juan	Pérez Martínez	NULL	NULL	juan.png	X1234567L
3	Ana	López Sánchez	a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p	2024-11-29 12:00:00	ana.png	Y1234567M
4	Pedro	Gómez Ruiz	q1w2e3r4t5y6u7i8o9p0a1s2d3f4g5h	2024-11-29 12:00:00	pedro.png	98765432A
5	Lucía	Martín Gómez	z9y8x7w6v5u4t3s2r1q0p9o8n7m6l5k	2024-11-29 12:00:00	avatar_default.png	Z7654321B
6	Carlos	Díaz Fernández	x1x2x3x4x5x6x7x8x9x0x1x2x3x4x5x6	2024-11-29 12:00:00	carlos.png	T1234567C

En la base de datos en el campo **hashp** está almacenada contraseña encriptada que figura en la tabla.

id	email	hashp	nombre	apellidos	imagen	confirmado	token	token_expira_a	telefono	nif_nie	fecha_nacimiento	rol	creado_a
1	maria.garcia@example.com	\$2y\$10\$7h6.xzwsZwuz0g2KFxOk3evVMuV3S2PQKn1SVh0NXpT...	María	García López	maria.png	1	NULL	NULL	600123456	12345678Z	1990-05-15	miembro	2024-11-27 11:49:25
2	juan.perez@example.com	\$2y\$10\$JRQ2v0kK55QH.lrZLNdOqeKi5rJ5t1w9ZGGe0OzMud...	Juan	Pérez Martínez	juan.png	1	NULL	NULL	600654321	X1234567L	1985-03-22	administrador	2024-11-27 11:49:25
3	ana.lopez@example.com	\$2y\$10\$q4vZFevSYJeJwXWwvg8.reQWum8zi64bOdpy2kwfui0...	Ana	López Sánchez	ana.png	NULL	a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p	2024-11-28 11:49:25	600987654	Y1234567M	1993-07-10	miembro	2024-11-27 11:49:25
4	pedro.gomez@example.com	\$2y\$10\$21.WBF5Llr6OXPHLmen8sOYPcME3iKVA4sUAbds5gGQ...	Pedro	Gómez Ruiz	pedro.png	NULL	q1w2e3r4t5y6u7i8o9p0a1s2d3f4g5h	2024-11-28 11:49:25	600111222	98765432A	1992-02-28	miembro	2024-11-27 11:49:25
5	lucia.martin@example.com	\$2y\$10\$31AkB7nMXExrmDSSYP.AuOrtKVj5vSzkB3MkRSTRNl...	Lucía	Martín Gómez	avatar_default.png	NULL	z9y8x7w6v5u4t3s2r1q0p9o8n7m6l5k	2024-11-28 11:49:25	600333444	Z7654321B	2000-11-05	miembro	2024-11-27 11:49:25
6	carlos.diaz@example.com	\$2y\$10\$vsBiuXo9iScPsonzfNkpYuUj6MLn9ffw1EuQoWP8kAM...	Carlos	Díaz Fernández	carlos.png	NULL	x1x2x3x4x5x6x7x8x9x0x1x2x3x4x5x6	2024-11-28 11:49:25	600555666	T1234567C	1995-08-18	miembro	2024-11-27 11:49:25

Ejemplo:

1. Generar el token original:

El servidor genera un token aleatorio para validar el correo electrónico, por ejemplo:

```
$token = bin2hex(random_bytes(32)); // Genera un token aleatorio
```

Este token se almacena en la base de datos junto con la fecha de expiración del token.

2. Encriptar el token:

Luego, el token generado se encripta utilizando password_hash() antes de enviarlo en la URL:

```
$hashedToken = password_hash($token, PASSWORD_DEFAULT);
```

3. URL con token encriptado:

El servidor genera la URL que incluirá el ID del usuario y el token encriptado en la URL. Supongamos que el ID del usuario es 123 y el hash generado para el token es:

```
$idUserario = 123;
```

```
$hashedToken = '$2y$10$VvBd6nJlhZrQ5eqZyZMkFsh5g1oZxJkHrF51v.vtq9Ex7e57RbZGm'; // Ejemplo de hash
```

La URL que se enviaría por correo al usuario sería algo como:

```
http://mi-sitio.com:8080/index.php?controlador=validar&id=123&token=\$2y\$10\$VvBd6nJlhZrQ5eqZyZMkFsh5g1oZxJkHrF51v.vtq9Ex7
```

La función `bin2hex(random_bytes(32))`

Combina dos funciones criptográficamente seguras para generar un token aleatorio y convertirlo en un formato legible.

- `random_bytes(32)`

Esta función genera una cadena de bytes aleatorios, con las siguientes características:

- **Longitud:** Se generan 32 bytes (256 bits) de datos aleatorios.
- **Calidad criptográfica:** Utiliza un generador de números aleatorios criptográficamente seguro, lo que significa que los valores son prácticamente impredecibles, incluso si un atacante conoce partes del sistema.
- **Propósito:** Se usa para crear valores únicos y seguros, ideales para contraseñas, tokens de sesión, claves secretas, etc.

- `bin2hex()`

Esta función convierte una cadena binaria en una representación hexadecimal. Cada byte (8 bits) se traduce a dos caracteres hexadecimales (0-9, a-f), lo que la hace mucho más legible y adecuada para ser usada en URL, formularios o bases de datos.

Comparar fechas:

```
// Comparar la fecha actual con la fecha de expiración que está guardada en la BD
$fechaExpiracion = new DateTime('2024-12-31 23:59:59'); //Fecha de expiración del token
$fechaActual = new DateTime(); // Fecha y hora actual
if ($fechaActual > $fechaExpiracion) {
    echo "El token ha expirado. ";
}
```

Pasos para implementar la validación

1. Recoger los parámetros de la URL

- Obtén el id y el token encriptados pasados como parámetros GET.

2. Validar el formato de los parámetros

- Asegúrate de que id sea un número entero y token encriptado tenga el formato esperado (`strlen(token)==60`).

3. Buscar en la base de datos

- Usa el **id** para localizar el registro en la base de datos.
- Confirma que el **token encriptado y el token original de la BD coinciden**.
- Verifica que aún no ha sido confirmado (`confirmado == 0`).
- Verifica si el token ha caducado comparándolo con la fecha y hora actual.

4. Actualizar el registro

- Si la validación es correcta, marca al usuario como confirmado (`confirmado = 1`), elimina el token (`token=NULL`)
- Indica al usuario que ya está dado de alta en la aplicación y muestra el **login**.

5. Manejar errores

- Si no se encuentra el registro o el token es inválido, muestra un mensaje adecuado. Nunca muestres el token, solo indica que ha caducado y dale la opción de solicitar uno nuevo.

Obtener un URL genérica, sirve para cualquier servidor donde esté alojada la aplicación

```
// Generar un token y guardarlo en la BD con el resto de los datos del usuario
$token = bin2hex(random_bytes(32));
// Debo obtenerlo con el método lastInsertId() cuando guardo el usuario en la BD
$idUsuario = '123';
// Encriptar el token
$hashedToken = password_hash($token, PASSWORD_DEFAULT);
// Obtener el esquema (http o https)
$esquema = $_SERVER['REQUEST_SCHEME'];
// Obtener el host
$host = $_SERVER['HTTP_HOST']; //Por ejemplo: localhost o 127.0.0.1
// Obtener el puerto si es necesario
$puerto = $_SERVER['SERVER_PORT'];
// Si el puerto no es 80 (para http) ni 443 (para https), lo añadimos a la URL
if (($esquema === 'http' && $puerto != 80) || ($esquema === 'https' && $puerto != 443)) {
    $host .= ':' . $puerto; // Añadimos el puerto si no es el predeterminado
}
// Obtener el nombre del script actual y la ruta a la carpeta de la aplicación
$script = dirname($_SERVER['SCRIPT_NAME']); // Obtenemos ruta relativa de la carpeta
// Generar la URL completa, combinando todos los elementos
$url = $esquema://$host$script/index.php?controlador=validar&id=$idUsuario&token=$hashedToken";
echo "La URL generada para validar el correo es: $url";
// Codificar toda la URL
$urlEncoded = urlencode($url);
echo "<br>La URL codificada es: $urlEncoded";
```

Sistema de Gestión de Usuarios con Registro, Inicio de Sesión y Recuperación de Contraseña

Implementar una aplicación web para gestionar el registro, inicio de sesión y recuperación de contraseñas de los usuarios. La aplicación debe utilizar base de datos `registros.sql` que almacena información personal y de autenticación de los usuarios. Además, debe incluir un sistema de roles (administrador y miembro) y permitir a los usuarios subir una imagen de perfil.

Requisitos Funcionales:

1. Registro de Usuarios:

El sistema debe permitir a los usuarios registrarse proporcionando:

- Nombre (campo obligatorio).
- Apellidos (campo obligatorio).
- Dirección de correo electrónico única (formato válido, obligatorio).
- Teléfono (formato válido, **no** obligatorio).
- NIF/NIE único (formato válido, obligatorio).
- Fecha de nacimiento (campo obligatorio, mayor de 16 años).
- Contraseña (mínimo 8 caracteres, debe coincidir con la repetición).
- Rol (por defecto será **miembro**, solo los administradores podrán cambiar este valor).
- Imagen de perfil (opcional, por defecto se asignará [avatar_default.png](#) si no se sube una imagen).

Validaciones en el Registro:

- Todos los campos obligatorios deben ser completados.
- Validar formato de correo, NIF/NIE y teléfono.
- La contraseña debe ser segura (mínimo 8 caracteres, al menos una letra y un número).
- La dirección de correo y el NIF/NIE deben ser únicos en la base de datos.
- La fecha de nacimiento debe garantizar que el usuario tiene más de 16 años.

2. Inicio de Sesión:

El sistema debe permitir a los usuarios iniciar sesión proporcionando:

- Dirección de correo electrónico.
- Contraseña.

Comportamiento:

- Validar que el correo existe en la base de datos.
- Comparar la contraseña con el hash almacenado.
- Verificar que la cuenta esté confirmada antes de permitir el acceso.

Si las credenciales son incorrectas, mostrar un mensaje de error.

3. Confirmación de Correo Electrónico:

- Generar un token único para cada nuevo registro.
- Al registrar un usuario, enviar un correo electrónico con un enlace para confirmar su cuenta.
- El enlace debe contener el id del usuario, el token encriptado y caducar después de 24 horas.

Validaciones:

- Verificar que el token sea válido y no haya expirado.
- Cambiar el estado del campo confirmado a 1 en la base de datos.
- Borrar el token y la fecha de expiración

4. Recuperación de Contraseña:

- Permitir a los usuarios solicitar un enlace de recuperación de contraseña ingresando su correo electrónico.
- Generar un nuevo token único y enviarlo por correo.
- El enlace de recuperación debe caducar en 24 horas.

Validaciones:

- Verificar que el correo proporcionado esté registrado.
- Permitir al usuario establecer una nueva contraseña siempre que sea válida.

5. Subida de Imagen de Perfil:

- Permitir a los usuarios subir una imagen de perfil durante el registro.
- Validar el tipo (.png y .jpg) y tamaño del archivo (solo imágenes de hasta 2 MB).

6. Mensajes de Error y Éxito:

- Mostrar mensajes claros y específicos para cada tipo de error (por ejemplo, "El correo ya está registrado", "El token ha expirado", etc.).
- Confirmar las acciones exitosas, como "Registro completado con éxito" o "Correo de recuperación enviado".

7. Descripción Adicional: Menú Personalizado tras Validación

Cuando un usuario inicia sesión correctamente, se mostrará un menú personalizado que incluirá:

- Imagen de Perfil: Mostrará la imagen que el usuario haya subido durante el registro o, si no se subió ninguna, la imagen predeterminada (avatar_default.png).
- Nombre: Aparecerá el nombre del usuario.
- Rol: Indicará si el usuario es "miembro" o "administrador".

Opciones del Menú: El contenido del menú dependerá del rol:

Miembro:

- Ver perfil.
- Editar perfil.
- Cerrar sesión.

Administrador:

- Ver perfil.
- Editar perfil.
- Gestionar usuarios.
- Cerrar sesión.

Validación de NIF

El formato general del NIF es:

Ocho números seguidos de una letra final.

Ejemplo: 12345678Z

1. Reglas del NIF:

- Debe tener exactamente 9 caracteres.
- Los primeros 8 deben ser dígitos.
- La última posición es una letra que se calcula en función del resto de la división de los 8 dígitos entre 23.

2. Cálculo de la letra del NIF:

- Hay un array de letras en el siguiente orden:

TRWAGMYFPDXBNJZSQVHLCKE

- Se toma el resto de dividir el número entre 23.
- La letra correspondiente es la que ocupa esa posición en el array.

Validación de NIE

El formato general del NIE es:

Una letra inicial (X, Y, Z) seguida de 7 números y una letra final.

Ejemplo: X1234567L

1. Reglas del NIE:

- La primera letra indica el grupo de extranjeros:
 - X: Se trata como un 0.
 - Y: Se trata como un 1.
 - Z: Se trata como un 2.
- Sustituir la primera letra por su valor numérico y calcular como un NIF.
- La letra final sigue las mismas reglas que el NIF.

2. Cálculo para el NIE:

- Convertir la letra inicial (X, Y, Z) a su valor numérico (0, 1, 2).
- Calcular la letra final como en el NIF.